

Arbitrage Scanner Web App Architecture

<https://gemini.google.com/app/633110bf280cd98>

User prompt: I need you to help me plan out the architecture of an arbitrage scanner web app

Response: A real-time arbitrage scanner—whether for sports betting or cryptocurrency—relies on a high-speed data pipeline that ingests, normalizes, and compares odds or order books before lines adjust. Latency is the primary enemy; an arbitrage opportunity often lasts only seconds to minutes.

1. Data Ingestion Layer

- **Purpose:** Fetch real-time odds or order book prices from multiple platforms simultaneously.
- **Technologies:** Go, Rust, or Python (asyncio) for managing high-concurrency network requests.
- **Execution:**
 - **WebSockets:** The primary method for real-time, low-latency updates. Essential for crypto exchanges and premium sports APIs.
 - **REST API Polling:** The fallback for platforms without WebSockets. Use distributed task queues (e.g., Celery, BullMQ) to maximize polling frequency without hitting rate limits.
 - **Data Aggregators:** To bypass building dozens of individual integrations initially, route through aggregators like SharpAPI for sports or CCXT for crypto.

2. Normalization Engine

- **Purpose:** Standardize disparate data formats into a single, unified schema before comparison.
- **The Challenge:** Fixing naming mismatches (e.g., "LA Lakers" on DraftKings vs. "LAL" on Pinnacle) and standardizing odds formats (fractional vs. decimal vs. American).
- **Execution:** Pass incoming data through a central mapping database. If an exact match isn't found, use fuzzy string matching (like Levenshtein distance) to flag the discrepancy for manual review. Without perfect normalization, the scanner will generate false positives.

3. Arbitrage Detection Engine

- **Purpose:** The core mathematical logic that identifies profitable discrepancies.
- **Execution:**
 - **Sports Logic:** Calculates implied probabilities. If the combined implied probability of all mutually exclusive outcomes across different bookmakers drops below 100%, an arbitrage exists.
 - **Crypto Logic:** Compares the highest Bid on Exchange A with the lowest Ask on Exchange B, explicitly deducting all withdrawal, network, and maker/taker fees.
- **Output:** Generates a standardized "Arbitrage Event" payload detailing the platforms, prices, expected profit margin, and exact stake ratios required to guarantee the profit.

4. Storage & Caching Layer

- **In-Memory Cache (Redis):** The most critical storage component. It holds the absolute latest state of all order books or odds. The detection engine constantly reads and writes to RAM to avoid disk I/O latency.
- **Relational Database (PostgreSQL):** Stores persistent data such as user accounts, subscription tiers, filter settings, historical arbitrage events, and entity mapping dictionaries.

5. API Gateway & Push Services

- **Purpose:** Serve the frontend and manage active user sessions.
- **Technologies:** Node.js (Express) or Python (FastAPI).
- **Real-Time Push:** Use Server-Sent Events (SSE) or WebSockets to push new Arbitrage Event payloads from the backend to active web clients instantly. Because arbs disappear quickly, relying on the frontend to poll the database is too slow.

This frontend requires a dense, tabular layout to display all variables simultaneously:

6. Frontend Application

- **Purpose:** The user-facing dashboard for filtering and acting on opportunities.
- **Technologies:** React, Next.js, or Vue.
- **Core UI:** A live, auto-updating data grid. It must include a built-in calculator where users can input their total capital for a specific trade and instantly see the exact stake splits required for each platform to execute the arb perfectly.

Development Note: If you are building this for decentralized crypto exchanges (DeFi), optimizing for compute units and priority fees on the blockchain becomes just as critical as the web architecture itself. For a practical walkthrough, The Blockchain Coders recently released a [source-code tutorial on building a DeFi scanner](#).

Sources

1. Build an Arbitrage Scanner - API Guide - SharpAPI

User prompt: Look I was thinking of having it in such a way that we use South African bookmakers or rather bookmakers that you can bet on as a South African

Response: Pivoting to a South African-focused arbitrage scanner changes the technical requirements significantly. Unlike international markets (like the US or UK) where dozens of clean, real-time odds APIs exist, the South African betting market is far more closed.

To build this for SA platforms like Hollywoodbets, Betway ZA, Supabets, World Sports Betting (WSB), and Sportingbet ZA, your architecture must heavily index on **data scraping and local server infrastructure**.

Here is how the architecture must adapt for the South African landscape:

1. The Ingestion Layer: Scraping vs. APIs

South African bookmakers rarely offer public developer APIs. You will have to extract odds yourself using a combination of methods:

- **Reverse-Engineering WebSockets:** Platforms like Sportingbet ZA and Betway often use WebSockets to push live odds to their frontend. You can inspect network traffic, grab the WebSocket URL and payload structure, and connect your backend directly to their data stream.
- **Headless Browsers (Puppeteer/Playwright):** For older platforms (like the legacy Hollywoodbets interface), you may need to run headless browsers to render the DOM and scrape the odds. This is resource-heavy, so you should route traffic through residential proxy IP addresses to avoid getting blocked by anti-bot protections like Cloudflare.
- **Niche Aggregators:** Some global odds APIs (like OddsPapi or The Odds API) do cover international brands operating in SA (e.g., Sportingbet, 10bet, Betfred). Use these where possible to reduce your scraping load.

2. South African Entity Normalization

Your normalization engine will be the hardest part to build. A 5-minute latency in matching teams means the arbitrage opportunity disappears. You must map local naming conventions strictly:

- **Soccer (PSL):** "Kaizer Chiefs" on Betway vs. "Chiefs" on Hollywoodbets vs. "K. Chiefs" on WSB.
- **Rugby (Currie Cup/URC):** "Bulls" vs "Vodacom Bulls".
- **Solution:** Use a PostgreSQL database to maintain a strict dictionary of team and league names. Run incoming scraped data against this dictionary. If a match fails, use fuzzy string matching (Levenshtein distance) to automatically link them if the match score is above 90%, or flag it for manual review.

3. Latency & Infrastructure (The af-south-1 Advantage)

Because arbitrage windows are tiny (often under 60 seconds), network latency matters.

- **Host Locally:** Deploy your backend, Redis cache, and scrapers in AWS Cape Town (af-south-1) or Azure South Africa North. This gives you single-digit millisecond ping times to local bookmaker servers, giving you a millisecond advantage over international competitors scraping SA books from Europe.
- **Redis Cache:** Store the live odds in RAM (Redis) rather than a database. Your detection engine should constantly scan Redis for mispriced lines across the SA books.

4. The "Last Mile" Execution (FICA Limits)

In international markets, premium arbitrage scanners offer "auto-betting" via APIs. **You cannot easily do this in South Africa.**

- **FICA Regulations:** Every SA betting account is strictly tied to a verified individual (ID and Proof of Address). Bookmakers aggressively flag and ban accounts using automated bots to place bets.
 - **The UI Solution:** Your React frontend should act strictly as an **Alert Dashboard & Calculator**. Push the arbitrage events to the user via Server-Sent Events (SSE). The user clicks the alert, your UI calculates the exact ZAR split required based on their bankroll, and the user manually places the bets on their dual screens or mobile app.
 - **Deep Linking:** Where possible, construct direct URL links to the specific bet slip (e.g., a direct link to the Chiefs vs. Pirates market on Supabets) so the user just has to click, input the ZAR amount, and confirm.
-

User prompt: What are the best practices and proxy setups to avoid getting IP banned while scraping sports betting sites?

Response: Sportsbooks protect their odds using advanced Web Application Firewalls (WAFs) like Cloudflare, Akamai, and DataDome. Scraping them successfully requires bypassing multiple layers of bot detection that start before your script even loads the page.

1. Proxy Selection and Rotation Strategy

Your proxy setup is the first line of defense against IP reputation checks.

- **Avoid Datacenter Proxies:** IPs from AWS, DigitalOcean, or Google Cloud have high fraud scores and are instantly blocked by betting sites.
- **Use Mobile Proxies (The Gold Standard):** Cellular networks use Carrier-Grade NAT (CGNAT), meaning hundreds of real mobile users share a single IP address. Betting sites are highly reluctant to ban mobile IPs because they risk blocking legitimate customers.
- **Residential Proxies:** If mobile proxies are cost-prohibitive, use high-quality residential proxies that route traffic through real household ISPs.
- **Session Management (Sticky IPs):** Do not rotate your IP on every single request. If a sportsbook sees a user session changing IPs mid-navigation, it triggers a bot flag. Configure your proxy provider to use "sticky sessions" that hold the same IP for 5 to 10 minutes before rotating.

2. Defeating TLS and Browser Fingerprinting

Standard HTTP clients (like Python's `requests` or Node's `axios`) will result in an instant 1020 Access Denied or 403 Forbidden error before Cloudflare even reads your headers.

- **TLS Handshake (JA3/JA4):** Firewalls analyze the exact cipher suites and extensions your script uses during the initial connection. A Python script has a distinct TLS fingerprint compared to Google Chrome.
- **The Solution:** You must spoof your TLS fingerprint to match a legitimate browser. Use impersonation libraries like `curl-impersonate`, `tls-client` in Python, or a specialized scraping API that handles TLS patching natively.

3. Headless Browsers and Stealth

If you need to execute JavaScript to bypass Cloudflare's Turnstile challenges or render dynamic odds:

- **Stealth Plugins:** If using Puppeteer or Playwright, standard headless modes are easily detected. You must use extensions like `puppeteer-extra-plugin-stealth` to mask automated variables (like the `navigator.webdriver` flag).
- **Hardware Fingerprinting:** WAFs will draw invisible graphics on the page to test how your machine renders them. Ensure your headless browser provides consistent, realistic Canvas and WebGL hardware fingerprints.

4. Behavioral Mimicry and Rate Limiting

Scraping at maximum speed is the fastest way to burn your proxy pool.

- **Introduce Jitter:** Never send requests at exact, static intervals. Implement randomized delays (e.g., 2 to 5 seconds) between page navigations.
- **Human Navigation Patterns:** If rendering the DOM, simulate realistic mouse movements, scroll events, and clicks before extracting the odds data.
- **Concurrency Limits:** Keep concurrent requests low per IP. Sending 20 simultaneous requests from a single residential IP is physically impossible for a human and will result in a ban.

5. Target Internal APIs (XHR Interception)

The most efficient way to scrape sportsbooks is to bypass the HTML entirely.

- **Intercept JSON Data:** Most modern betting sites load their odds dynamically via internal APIs (XHR/Fetch requests) after the shell of the page loads.
- **Execution:** Open your browser's Network tab, find the endpoint returning the odds in JSON format, and copy the request headers (including cookies and authorization tokens). Replicate this specific HTTP request in your scraper to get clean data instantly, reducing the need for heavy headless browsers.

Sources

1. [How to Bypass Cloudflare When Web Scraping in 2026 - ScrapeUp](#)
2. [Web Scraping Part5: How to Bypass Cloudflare. - Medium](#)
3. [How to Bypass Cloudflare With Scraping \(2026\) - Browserless](#)
4. [Web Scraping Without Bans: Best Practices \(Updated June 2026\)](#)
5. [How to scrape Sports Betting sites in 2026: 3 methods - Roundproxies](#)

User prompt: How do I implement TLS impersonation using Python to bypass Cloudflare 1020 errors?

Response: Standard HTTP clients like Python's `requests` or `aiohttp` rely on the built-in `ssl` module, which sends a `TLS ClientHello` packet that is instantly recognizable as a bot. Cloudflare reads this signature (known as a JA3/JA4 fingerprint) and returns a 1020 Access Denied error before your script even requests the page.

To bypass this, you must compile your HTTP requests using the exact TLS and HTTP/2 network stacks of a real web browser. In Python, the most efficient way to achieve this is using `curl_cffi`, a library built on top of `curl-impersonate`.

Step 1: Installation

Install the package via pip. It includes pre-compiled binaries, so you do not need to install `curl` locally.

```
pip install curl_cffi --upgrade
```

Step 2: Basic Synchronous Impersonation

The `curl_cffi.requests` module acts as a drop-in replacement for the standard `requests` library. By setting the `impersonate` parameter, the library automatically alters the TLS ciphers, ALPN protocols, and HTTP/2 frame order to exactly match the target browser.

```
from curl_cffi import requests

# Impersonate a specific version of Chrome
# This overrides the Python TLS footprint completely
response = requests.get(
    "https://example-sportsbook.co.za/api/odds",
    impersonate="chrome124",
    headers={
        # Ensure your User-Agent exactly matches the impersonated version
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0
    }
)

print(response.status_code) # Should return 200 instead of 403/1020
```

Step 3: High-Speed Async Implementation

For a real-time arbitrage scanner, synchronous requests are too slow. `curl_cffi` provides an `AsyncSession` that works perfectly with `asyncio` to handle multiple bookmakers concurrently.

```
import asyncio
from curl_cffi.requests import AsyncSession

async def fetch_bookmaker_odds(urls):
    # Use a session to maintain connection pooling and cookies
    async with AsyncSession(impersonate="chrome124") as session:
        tasks = []
        for url in urls:
            # Add tasks to the event loop
            tasks.append(session.get(url))

        # Execute all requests concurrently
        responses = await asyncio.gather(*tasks)

        for response in responses:
            print(f"Status: {response.status_code} | URL: {response.url}")
            # process_odds(response.json())

urls_to_scrape = [
    "https://api.bookmaker1.co.za/v1/markets",
    "https://api.bookmaker2.co.za/v1/markets"
]

asyncio.run(fetch_bookmaker_odds(urls_to_scrape))
```

Step 4: Routing Through Proxies

TLS impersonation alone will not save you if your IP address has a bad reputation. Cloudflare analyzes both the JA3 fingerprint and the IP classification simultaneously. You must route your impersonated requests through residential or mobile proxies.

```
from curl_cffi import requests

# Format: http://username:password@ip:port
proxy_url = "http://USER:PASS@proxy.provider.com:7777"

proxies = {
    "http": proxy_url,
    "https": proxy_url
}

response = requests.get(
    "https://example-sportsbook.co.za/api/odds",
    impersonate="chrome124",
    proxies=proxies,
    timeout=10 # Proxies introduce latency; set a reasonable timeout
)
```

Critical Rules for Success

- **Version Matching:** Cloudflare cross-references your TLS signature with your HTTP headers. If you set `impersonate="chrome124"` but pass a Firefox User-Agent, Cloudflare will flag the mismatch and block you.

- **Header Ordering:** Browser HTTP/2 headers are sent in a specific order. Standard Python requests scrambles this order. `curl_cffi` handles the ordering natively, so avoid manually overriding headers unless necessary.
- **JavaScript Challenges:** TLS impersonation bypasses network-level WAF checks (1020 errors). However, if the site forces a Turnstile CAPTCHA or a JavaScript execution challenge (often resulting in a 403 or 503 status code), `curl_cffi` will fail because it does not render the DOM or run JavaScript. If that occurs, you must switch back to a patched headless browser like Playwright with stealth plugins.

Sources

1. [How to Bypass Cloudflare When Web Scraping in 2026 - Scrapfly](#)
2. `curl_cffi`'s documentation — [curl_cffi documentation](#)
3. [How to Bypass Cloudflare. Intro | by Noorsimar Singh - Medium](#)
4. [How to Bypass Cloudflare When Web Scraping \(2026\) - Use Apify](#)

User prompt: How do I configure Playwright with stealth plugins to bypass Cloudflare JavaScript challenges?

Response: When TLS impersonation via `curl_cffi` fails because Cloudflare triggers a 5-second JS challenge, a Turnstile CAPTCHA, or an invisible bot-management script, you must switch to a headless browser. However, a vanilla Playwright instance is instantly flagged because it leaks dozens of automation fingerprints (like the `navigator.webdriver` flag, missing hardware features, and headless-specific Chrome flags).

To bypass this in Node.js (or Python), you must use stealth extensions and spoof human interactions. The industry standard for this is combining Playwright with Puppeteer's stealth ecosystem.

Step 1: Install the Stealth Ecosystem (Node.js)

The Playwright core library does not natively support plugins. You need the `playwright-extra` wrapper, which allows you to run Puppeteer stealth plugins inside Playwright.

```
npm install playwright playwright-extra puppeteer-extra-plugin-stealth
```

Step 2: Configure the Stealth Browser

The `puppeteer-extra-plugin-stealth` package automatically applies a massive suite of evasions before the page loads. It deletes the `webdriver` property, spoofs the `window.chrome` runtime object, masks WebGL vendor strings, and bypasses standard iframe detection.

```
const { chromium } = require('playwright-extra');
const stealth = require('puppeteer-extra-plugin-stealth');

// Apply the stealth plugin to the Chromium instance
chromium.use(stealth());

(async () => {
  // Crucial: Cloudflare often blocks 'headless: true'.
  // Always use headed mode when testing, and for production use Xvfb (virtual frame buffer)
  // to run headed mode on a headless Linux server.
  const browser = await chromium.launch({
    headless: false,
    args: [
      '--disable-blink-features=AutomationControlled', // Hides the "Chrome is being controlled" banner and in
      '--disable-web-security',
      '--no-sandbox'
    ]
  });

  // Set a realistic viewport and User-Agent.
  // The User-Agent MUST match the browser version Playwright downloaded.
  const context = await browser.newContext({
    viewport: { width: 1920, height: 1080 },
    userAgent: 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0'
  });

  const page = await context.newPage();

  // Proceed to the target sportsbook
  await page.goto('https://example-sportsbook.co.za', { waitUntil: 'domcontentloaded' });

  console.log("Page loaded. Cloudflare should be bypassed.");

  // await browser.close();
})();
```

Step 3: Evading Behavioral Detection (The "Human" Factor)

Cloudflare's Turnstile doesn't just look at browser flags; it analyzes *how* the page is navigated. If you instantly teleport the mouse to a button or navigate at robotic speeds, the invisible Turnstile challenge will fail.

You must introduce simulated human behavior *before* extracting the odds:

```
// A helper function for realistic delays
const delay = (ms) => new Promise(resolve => setTimeout(resolve, ms));

async function simulateHumanInteraction(page) {
  // 1. Randomize the initial load delay
  await delay(Math.floor(Math.random() * 2000) + 1500);

  // 2. Move the mouse on a randomized curve, not a straight line
  await page.mouse.move(
    Math.floor(Math.random() * 500) + 100,
    Math.floor(Math.random() * 500) + 100,
    { steps: 15 } // Breaks the movement into multiple frames
  );

  // 3. Scroll down slightly to trigger lazy-loaded odds tables
  await page.mouse.wheel(0, Math.floor(Math.random() * 400) + 200);

  await delay(1000);
}

// Inside your main script:
await page.goto('https://example-sportsbook.co.za/matches');
await simulateHumanInteraction(page);

// Now extract the odds after Cloudflare has validated the "human" session
const odds = await page.evaluate(() => {
  // DOM extraction logic here
});
```

Alternative for Python Developers

If your arbitrage backend is entirely in Python, you can achieve the exact same setup using the playwright-stealth port.

```
pip install playwright playwright-stealth
playwright install

from playwright.sync_api import sync_playwright
from playwright_stealth import stealth_sync

with sync_playwright() as p:
    browser = p.chromium.launch(
        headless=False,
        args=["--disable-blink-features=AutomationControlled"]
    )
    page = browser.new_page(
        user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.
        viewport={"width": 1920, "height": 1080}
    )

    # Apply stealth scripts to the page before navigating
    stealth_sync(page)

    page.goto("https://example-sportsbook.co.za")
    print(page.title())
    browser.close()
```

The Production Strategy for Arbitrage

Running headed Playwright instances is computationally expensive and slow—terrible for a time-sensitive arbitrage scanner. The best architecture is a **hybrid approach**:

1. Use the heavy, stealthy Playwright instance *only once* to load the site, solve the Cloudflare JS challenge, and acquire the session cookies (specifically the `cf_clearance` cookie).
2. Extract those cookies from Playwright.
3. Inject those cookies into your fast `curl_cffi` (Python) or `axios` (Node.js) HTTP client to scrape the raw JSON APIs directly at high speed until the clearance cookie expires (usually 30–120 minutes).

User prompt: How do I extract the `cf_clearance` cookie from Playwright and pass it into a fast HTTP client?

Response: To bypass Cloudflare heavy rendering for high-speed scraping, you can use Playwright solely to solve the initial security challenge and capture the issued `cf_clearance` cookie. Once obtained, you export that cookie—along with the exact User-Agent string and proxy session—and pass it directly into an HTTP client like `curl_cffi`.

Cloudflare validates three core variables simultaneously: the `cf_clearance` cookie, the exact User-Agent string, and the client's egress IP address. If any of these change between Playwright and your HTTP client, Cloudflare invalidates the clearance token immediately.

Complete Python Implementation (playwright + curl_cffi)

Here is an end-to-end script that solves the challenge in Playwright, extracts the session state, and transfers it to curl_cffi:

```
import asyncio
from playwright.async_api import async_playwright
from curl_cffi import requests

# If using a proxy, it MUST be the exact same sticky proxy for both Playwright and curl_cffi
PROXY_URL = None # e.g., "http://username:password@proxy.provider.com:7777"
TARGET_URL = "https://example-sportsbook.co.za"
API_ENDPOINT = "https://example-sportsbook.co.za/api/v1/live-odds"

async def extract_cloudflare_session():
    async with async_playwright() as p:
        # Launch Playwright in headed mode (or via Xvfb) with automation flags disabled
        browser_args = ["--disable-blink-features=AutomationControlled"]

        launch_options = {
            "headless": False,
            "args": browser_args
        }
        if PROXY_URL:
            launch_options["proxy"] = {"server": PROXY_URL}

        browser = await p.chromium.launch(**launch_options)
        context = await browser.new_context(
            viewport={"width": 1920, "height": 1080}
        )
        page = await context.new_page()

        print("[1] Navigating to page to trigger Cloudflare challenge...")
        await page.goto(TARGET_URL, wait_until="domcontentloaded")

        # Wait until the cf_clearance cookie is set in the context
        print("[2] Waiting for cf_clearance cookie...")
        cf_cookie = None
        for _ in range(30): # Poll up to 30 seconds
            cookies = await context.cookies()
            cf_cookie = next((c for c in cookies if c["name"] == "cf_clearance"), None)
            if cf_cookie:
                break
            await asyncio.sleep(1)

        if not cf_cookie:
            await browser.close()
            raise Exception("Failed to acquire cf_clearance cookie within time limit.")

        # Extract all cookies formatted as a dictionary for curl_cffi
        cookie_dict = {c["name"]: c["value"] for c in cookies}

        # Extract the exact User-Agent string used by the Playwright browser instance
        user_agent = await page.evaluate("navigator.userAgent")

        print(f"[3] Successfully captured cf_clearance: {cf_cookie['value'][:15]}...")
        await browser.close()

        return cookie_dict, user_agent

def run_fast_api_requests(cookies, user_agent):
    print("[4] Initializing high-speed HTTP client with extracted session...")

    # Initialize a curl_cffi session with Chrome impersonation
    session = requests.Session(impersonate="chrome124")

    # Configure proxies if applicable
    if PROXY_URL:
        session.proxies = {"http": PROXY_URL, "https": PROXY_URL}

    # Pass the exact User-Agent and Cookies extracted from Playwright
    headers = {
        "User-Agent": user_agent,
        "Accept": "application/json, text/plain, */*",
        "Referer": TARGET_URL
    }

    # Execute high-speed requests against internal JSON APIs
    response = session.get(
        API_ENDPOINT,
```

```

        headers=headers,
        cookies=cookies,
        timeout=10
    )

    print(f"[5] API Response Status: {response.status_code}")
    if response.status_code == 200:
        print("Successfully bypassed Cloudflare! Data payload received.")
        # print(response.json())
    else:
        print(f"Failed. StatusTo successfully bypass Cloudflare's bot protection, the HTTP client must mirror the ex

```

Here is the complete Python implementation using `playwright` and `curl\_cffi`.

The Python Implementation

This script uses Playwright to navigate to the target, waits for Cloudflare to issue the clearance cookie, extracts

```

```python
import asyncio
from playwright.async_api import async_playwright
from curl_cffi import requests

If using a proxy, it MUST be the exact same proxy for both Playwright and curl_cffi
PROXY_URL = None # e.g., "http://username:password@proxy.provider.com:7777"
TARGET_URL = "https://example-sportsbook.co.za"
API_ENDPOINT = "[https://example-sportsbook.co.za/api/v1/live-odds](https://example-sportsbook.co.za/api/v1/live-odd

async def get_cloudflare_session():
 """Uses Playwright to solve the challenge and extract the cf_clearance cookie."""
 async with async_playwright() as p:
 # Launch headed (or via Xvfb on Linux) to avoid headless bot detection
 browser_args = ["--disable-blink-features=AutomationControlled"]
 launch_options = {"headless": False, "args": browser_args}

 if PROXY_URL:
 launch_options["proxy"] = {"server": PROXY_URL}

 browser = await p.chromium.launch(**launch_options)
 context = await browser.new_context(viewport={"width": 1920, "height": 1080})
 page = await context.new_page()

 print("[1] Navigating to target to trigger Cloudflare challenge...")
 await page.goto(TARGET_URL, wait_until="domcontentloaded")

 # Wait until the cf_clearance cookie is issued by the server
 print("[2] Waiting for cf_clearance cookie...")
 cf_cookie = None
 for _ in range(30): # Poll for up to 30 seconds
 cookies = await context.cookies()
 cf_cookie = next((c for c in cookies if c["name"] == "cf_clearance"), None)
 if cf_cookie:
 break
 await asyncio.sleep(1)

 if not cf_cookie:
 await browser.close()
 raise Exception("Timeout: Failed to acquire cf_clearance cookie.")

 # Format cookies into a dictionary for curl_cffi
 cookie_dict = {c["name"]: c["value"] for c in cookies}

 # Extract the exact User-Agent string generated by the Playwright browser
 user_agent = await page.evaluate("navigator.userAgent")

 print(f"[3] Successfully captured cf_clearance: {cf_cookie['value'][:15]}...")
 await browser.close()

 return cookie_dict, user_agent

def fetch_odds_high_speed(cookies, user_agent):
 """Uses curl_cffi to fetch JSON APIs at high speed using the Playwright session."""
 print("[4] Initializing curl_cffi with extracted session state...")

 # Initialize a session impersonating a Chrome version closely matching your Playwright build
 session = requests.Session(impersonate="chrome124")

 if PROXY_URL:

```

```

 session.proxies = {"http": PROXY_URL, "https": PROXY_URL}

Pass the exact User-Agent. This is strictly required by Cloudflare.
headers = {
 "User-Agent": user_agent,
 "Accept": "application/json, text/plain, */*",
 "Referer": TARGET_URL
}

Execute high-speed requests against the internal XHR/JSON APIs
response = session.get(
 API_ENDPOINT,
 headers=headers,
 cookies=cookies,
 timeout=10
)

print(f"[5] API Response Status: {response.status_code}")
if response.status_code == 200:
 print("Success! Bypassed Cloudflare and received API payload.")
 # print(response.json())
else:
 print(f"Failed. Received status {response.status_code}.")

async def main():
 try:
 cookies, user_agent = await get_cloudflare_session()

 # Once the session is extracted, you can loop this fast HTTP request
 # until the cf_clearance cookie expires (usually 30 to 120 minutes).
 fetch_odds_high_speed(cookies, user_agent)
 except Exception as e:
 print(f"Error: {e}")

if __name__ == "__main__":
 asyncio.run(main())

```

## Critical Architecture Rules

- 1. The Expiration Loop:** The `cf_clearance` cookie is not permanent. It typically lasts between 30 and 120 minutes depending on the sportsbook's security policy. Your backend must catch 403 Forbidden or 1020 errors from `curl_cffi`, pause the data scraping loop, and spin up the Playwright container to refresh the clearance cookie.
- 2. Proxy Bleed:** If you are using residential rotating proxies, ensure you are using a **sticky session ID**. If your IP address rotates between the time Playwright gets the cookie and `curl_cffi` makes its first request, Cloudflare will reject the token.
- 3. Impersonate Version Alignment:** The `impersonate="chrome124"` parameter in `curl_cffi` should roughly align with the Chromium version your Playwright installation downloaded. If Playwright uses Chrome 124 but `curl_cffi` impersonates Safari or an outdated Chrome version, the TLS fingerprint mismatch will trigger a block.

*User prompt: How do I build the data normalization engine to standardize the JSON odds data I am scraping?*

Response: The normalization engine acts as high-speed middleware. It ingests chaotic, inconsistently formatted JSON payloads from your various South African scrapers and transforms them into a single, strict schema that your arbitrage math engine can process blindly.

## 1. Define the Universal Schema

Before normalizing data, you must define the strict output format. Every scraper, regardless of the bookmaker, must map its payload to this exact structure.

```

{
 "event_id": "standardized-uuid-or-hash",
 "sport": "soccer",
 "league": "south_africa_premier_league",
 "home_team": "Kaizer Chiefs",
 "away_team": "Orlando Pirates",
 "start_time": "2026-10-24T15:00:00Z",
 "bookmaker": "betway_za",
 "markets": {
 "moneyline": {
 "home_odds": 2.10,
 "away_odds": 3.40,
 "draw_odds": 3.10
 }
 }
}

```

## 2. Format Normalization (Prices and Markets)

Bookmakers use different terminologies and odds formats. This layer standardizes the mathematics and the market categories before touching team names.

- **Odds Conversion:** Convert all incoming odds to Decimal format. If a scraper pulls UK fractional odds (e.g., 5/2), convert it immediately:  $(\text{Numerator} / \text{Denominator}) + 1 = 3.50$ .
- **Market Mapping:** Create a static dictionary mapping bookie-specific market names to your universal keys.
  - "1X2" (Betway) → "moneyline"
  - "Match Result" (Hollywoodbets) → "moneyline"
  - "Total Match Goals Over/Under" → "totals"

## 3. The Entity Resolution Engine

This is the most complex component. "Kaizer Chiefs" on one bookmaker might be "Chiefs" or "K. Chiefs" on another. If the engine fails to recognize they are the same team, you miss the arbitrage opportunity.

### The Three-Tier Matching Pipeline

Because fuzzy string matching is computationally expensive, you cannot run it on every single incoming JSON payload. You must use a tiered approach:

1. **Tier 1: Redis Exact Match (Sub-millisecond):** Maintain an in-memory dictionary where the key is a composite of `league_name:scraped_team_name` and the value is your `universal_team_name`. If `psl:k. chiefs` is in the cache, map it instantly and move on.
2. **Tier 2: Algorithmic Fuzzy Matching (10-50 milliseconds):** If the exact match fails, pass the string to a fuzzy matching algorithm. In Python, the industry standard is **RapidFuzz**. It is significantly faster than older libraries like FuzzyWuzzy and handles large-scale string comparisons efficiently.
  - Use `fuzz.token_sort_ratio`. This evaluates the strings regardless of word order (e.g., matching "FC Cape Town" to "Cape Town FC" perfectly).
  - If the RapidFuzz similarity score is **above 88%**, automatically accept the match, write the new variation to the Redis Exact Match cache for next time, and proceed.
3. **Tier 3: The Dead Letter Queue (Manual Review):** If the RapidFuzz score falls below your confidence threshold (e.g., 88%), do NOT process the odds. Route the JSON payload to a database queue. Your admin dashboard should have a UI where you manually link the unrecognized team to the master database. Once manually linked, it pushes to the Redis cache and will auto-match forever.

## 4. Normalizing Time and Identifiers

Timezones and event IDs will ruin your matching if not standardized.

- **Time Normalization:** Parse all incoming datetime strings (which may arrive in SAST, GMT, or Unix timestamps) and convert them to strict UTC ISO 8601 strings.
- **Creating the Event Hash:** To compare odds across bookmakers, you need a unique identifier for the game itself. Create an MD5 hash combining the normalized data: `hash(home_team + away_team + start_date_utc)`. This hash becomes the primary key your arbitrage detection engine uses to group the odds together.

### Sources

1. [GitHub - rapidfuzz/RapidFuzz: Rapid fuzzy string matching in Python](#)

*User prompt: How do I ensure the data normalization engine processes live, in-play JSON payloads without introducing too much latency?*

**Response:** When dealing with in-play (live) sports betting, odds change every few seconds. If your normalization engine takes 50 milliseconds to process a payload, the arbitrage opportunity might vanish before it reaches your detection engine.

To achieve ultra-low latency (sub-5ms) in your normalization pipeline, you must completely remove blocking I/O, heavy string manipulation, and standard library bottlenecks from the "hot path" (the main execution loop).

Here is how you architect the engine for real-time speed:

### 1. Ban Fuzzy Matching from the "Hot Path"

You absolutely **cannot** run RapidFuzz (or any string similarity algorithm) synchronously on live odds.

- **The Rule:** If an incoming live event or team name does not have an exact match in your cache, **drop the payload immediately** or route it to a background queue.
- **The Workflow:** Live events are almost always listed hours before kickoff. Your system should have already normalized the team names and generated the Event Hash during the "Pre-Match" phase using the slower fuzzy matching pipeline. When the game goes live, your engine only needs to look up the pre-calculated Event ID.

## 2. Ditch the Network: Use an L1/L2 Cache Architecture

Querying Redis for every incoming JSON payload introduces network latency (1–3ms per round trip, even in the same AWS region). For thousands of odds ticks per second, this is crippling.

- **L1 (Local Memory):** Load your entire entity mapping dictionary directly into your application's RAM (e.g., a native Python `dict` or a Go `map`) on startup. Lookups take nanoseconds.
- **L2 (Redis):** Use Redis solely as the central source of truth. When a new team is mapped in the background, publish the update via Redis Pub/Sub. Your normalization servers listen to this channel and instantly update their L1 local dictionaries without interrupting the main processing loop.

## 3. Replace the Standard JSON Library

Python's built-in `json` module is notoriously slow. You must swap it out for a Rust-based or C-based parser.

- **Use `orjson`:** It is the fastest JSON library for Python. It serializes and deserializes payloads up to 10x faster than the standard library and natively supports `datetime` objects and NumPy arrays.

```
import orjson

Deserialization is instantly faster
payload = orjson.loads(incoming_websocket_bytes)

Serialization
normalized_bytes = orjson.dumps(normalized_dict)
```

## 4. Optimize the Event Loop (`uvloop`)

If you are writing the ingestion and normalization engine in Python using `asyncio`, the default event loop will become a bottleneck when handling thousands of concurrent WebSocket messages.

- **The Solution:** Install and enable `uvloop`. It is a Cython-based drop-in replacement for the standard `asyncio` event loop, built on top of `libuv` (the same engine that powers Node.js). It makes Python's async networking 2-4x faster.

```
import asyncio
import uvloop

Call this at the very beginning of your application
asyncio.set_event_loop_policy(uvloop.EventLoopPolicy())
```

## 5. ID-Based Normalization (Integer vs. String)

String comparisons (e.g., `if team_name == "Kaizer Chiefs"`) are inherently slower than integer comparisons.

- Instead of passing the full string payload through the pipeline, hash the strings into integers as early as possible.
- Map the bookmaker's internal ID (which is usually an integer, e.g., `event_id: 10458392`) directly to your universal `event_id`. This means your normalizer doesn't even read the team names on live payloads—it just reads `10458392`, instantly maps it to your internal UUID, updates the odds, and pushes it to the math engine.

## 6. Consider Go or Rust for the Ingestion Microservice

If you push Python to its absolute limits and still experience garbage collection pauses or GIL (Global Interpreter Lock) bottlenecks, you should rewrite just this specific microservice in **Go**.

- Go's native concurrency (Goroutines and Channels) and compiled speed make it the industry standard for high-frequency trading and live odds ingestion. You can have Go ingest the WebSockets, normalize the JSON using native structs, and push the clean data to Redis, while your Python backend handles the slower database and user-facing API tasks.

*User prompt: How do I calculate multi-way arbitrage and implied probability once the data is normalized?*

**Response:** Arbitrage betting relies on exploiting situations where the combined implied probability of all mutually exclusive outcomes in a market falls below 100%. Because your normalization engine has already converted everything to decimal odds, the math to detect these opportunities and allocate stakes requires three specific formulas.

## 1. Implied Probability

First, convert the decimal odds for every outcome into a percentage representing the bookmaker's estimated probability.

$$IP = \frac{1}{\text{Decimal Odds}}$$

For example, if Betway ZA offers 2.60 on a Kaizer Chiefs win, the implied probability (*IP*) is  $1/2.60 = 0.3846$ , or 38.46%.

## 2. The Arbitrage Condition (Market Margin)

To check if an arbitrage exists, sum the implied probabilities of the highest available odds for every outcome across all bookmakers.

$$M = \sum_{i=1}^n IP_i = IP_1 + IP_2 + \dots + IP_n$$

- If  $M > 1.0$  (or 100%): The bookmakers have an edge (the "vig"). No arbitrage exists.
- If  $M < 1.0$  (or 100%): An arbitrage exists. The lower the number, the higher your profit margin.

### 3. The Stake Allocation Formula

Once an arb is detected, you must split your total ZAR bankroll so that the final payout is identical regardless of which outcome actually wins.

$$S_i = \frac{B \times IP_i}{M}$$

Where:

- $S_i$  = The specific stake to place on outcome  $i$
- $B$  = Your total betting bankroll
- $IP_i$  = The implied probability of outcome  $i$
- $M$  = The combined market margin calculated in Step 2

### Worked Example: 3-Way PSL Soccer Match

Assume you are scanning a 3-way (1X2) market for a Chiefs vs. Pirates match. Your engine finds the best odds across three different South African books. You have a total bankroll of **R1,000**.

Outcome	Best Bookmaker	Odds	Implied Probability ( $IP$ )	Stake Split ( $S_i$ )	Expected Payout
Chiefs Win	Supabets	2.60	$1/2.60 = 0.3846$	$(1000 \times 0.3846)/0.9733 = R395.15$	R1,027.39
Draw	Betway ZA	3.30	$1/3.30 = 0.3030$	$(1000 \times 0.3030)/0.9733 = R311.31$	R1,027.32
Pirates Win	Hollywoodbets	3.50	$1/3.50 = 0.2857$	$(1000 \times 0.2857)/0.9733 = R293.54$	R1,027.39

- **Total Market Margin ( $M$ ):**  $0.3846 + 0.3030 + 0.2857 = 0.9733$  (97.33%)
- **Total Staked:** R1,000
- **Guaranteed Profit:** ~R27.35 (2.73% ROI) (Note: Payouts differ by a few cents due to rounding)

### Python Implementation

Because your normalization engine outputs clean JSON, your detection microservice can run this calculation in fractions of a millisecond using native Python:

```
def calculate_arbitrage(odds_list: list[float], bankroll: float):
 # Calculate implied probabilities for all outcomes
 implied_probs = [1 / odds for odds in odds_list]

 # Calculate the total market margin
 market_margin = sum(implied_probs)

 # Check if an arb exists
 if market_margin < 1.0:
 # Calculate exactly how much to stake on each outcome
 stakes = [round((bankroll * ip) / market_margin, 2) for ip in implied_probs]

 # Calculate the guaranteed return and ROI
 guaranteed_return = bankroll / market_margin
 profit = guaranteed_return - bankroll
 roi_percentage = (profit / bankroll) * 100

 return {
 "arbitrage_found": True,
 "margin": round(market_margin, 4),
 "stakes": stakes,
 "expected_profit": round(profit, 2),
 "roi_percent": round(roi_percentage, 2)
 }

 return {"arbitrage_found": False}
```

```
Example usage for the 1X2 market above:
result = calculate_arbitrage([2.60, 3.30, 3.50], 1000.00)
```

**The Rounding Trap:** Most South African bookmakers only accept bets in whole Rands or to two decimal places. If your math engine tells the user to bet R395.15 but the sportsbook rounds it to R395.00, the arbitrage is no longer perfectly balanced. Your frontend dashboard must include a calculator that allows the user to round their stakes to whole numbers and instantly see the slightly skewed profit margins.

## Sources

1. [Arbitrage Betting Formula - Predict & Profit - RebelBetting](#)
2. [How to calculate arbitrage betting - Smarkets Help Centre](#)

*User prompt: How do I use Server-Sent Events (SSE) to push these calculated arbitrage alerts to my React dashboard instantly?*

Response: Server-Sent Events (SSE) are perfect for an arbitrage dashboard. Unlike WebSockets, which are designed for two-way communication (like a chat app), SSE is explicitly designed for a **one-way firehose** of data from the server to the client. It runs over standard HTTP, making it much easier to load-balance and secure behind firewalls.

Here is how you build the SSE pipeline using Python (FastAPI) on the backend and React on the frontend.

## 1. The Backend (FastAPI + Redis)

Your FastAPI server needs an endpoint that does not return a standard JSON response, but instead returns a `StreamingResponse`. This endpoint will subscribe to your Redis channel (where the Math Engine publishes found arbs) and instantly yield them to the open HTTP connection.

```
import asyncio
import json
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from fastapi.middleware.cors import CORSMiddleware
import redis.asyncio as redis

app = FastAPI()

Allow your React app to connect
app.add_middleware(
 CORSMiddleware,
 allow_origins=["http://localhost:3000"],
 allow_credentials=True,
 allow_methods=["*"],
 allow_headers=["*"],
)

Connect to the local Redis instance
redis_client = redis.from_url("redis://localhost:6379", decode_responses=True)

async def arb_event_generator():
 """Generator function that yields SSE formatted strings."""
 pubsub = redis_client.pubsub()
 await pubsub.subscribe("live_arbitrage_alerts")

 try:
 while True:
 # Listen for new messages pushed by the Math Engine
 message = await pubsub.get_message(ignore_subscribe_messages=True, timeout=1.0)

 if message:
 arb_data = message["data"]
 # SSE format strictly requires "data: {payload}\n\n"
 yield f"data: {arb_data}\n\n"
 else:
 # Send a keep-alive ping every few seconds to prevent timeouts
 yield ": keep-alive\n\n"

 await asyncio.sleep(0.1) # Yield control back to the event loop

 except asyncio.CancelledError:
 # Fired when the client closes their browser tab
 print("Client disconnected.")
 await pubsub.unsubscribe("live_arbitrage_alerts")

@app.get("/api/stream/arbitrage")
async def stream_arbitrage():
 """Endpoint that the React app connects to."""
 return StreamingResponse(
 arb_event_generator(),
```

```

 media_type="text/event-stream"
)

```

## 2. The Frontend (React Dashboard)

In your React application, you will use the native browser EventSource API. You do not need any external libraries for this.

```

import React, { useEffect, useState } from 'react';

const ArbitrageDashboard = () => {
 const [arbs, setArbs] = useState([]);
 const [connectionStatus, setConnectionStatus] = useState('Connecting...');

 useEffect(() => {
 // Open the persistent SSE connection
 const eventSource = new EventSource('http://localhost:8000/api/stream/arbitrage');

 eventSource.onopen = () => {
 setConnectionStatus('Connected (Live)');
 };

 // Fired every time the FastAPI server yields "data: {...}\n\n"
 eventSource.onmessage = (event) => {
 const newArb = JSON.parse(event.data);

 setArbs((prevArbs) => {
 // Keep only the 50 most recent arbs to prevent memory bloat
 const updatedArbs = [newArb, ...prevArbs];
 return updatedArbs.slice(0, 50);
 });
 };

 eventSource.onerror = (error) => {
 console.error('SSE Error:', error);
 setConnectionStatus('Disconnected - Retrying...');
 // EventSource automatically attempts to reconnect, but you can add custom logic here
 };

 // Cleanup function when the component unmounts
 return () => {
 eventSource.close();
 };
 }, []);

 return (
 <div className="p-6 bg-gray-900 text-white min-h-screen">
 <div className="flex justify-between items-center mb-6">
 <h1 className="text-2xl font-bold">Live Arbitrage Feed</h1>
 {connectionStatus}
 </div>

 <div className="grid gap-4">
 {arbs.length === 0 ? (
 <p className="text-gray-400">Waiting for opportunities...</p>
) : (
 arbs.map((arb, index) => (
 <div key={index} className="bg-gray-800 p-4 rounded border border-gray-700">
 <div className="flex justify-between">
 {arb.match_title}
 {arb.roi_percent}% ROI
 </div>
 </* Render your stake splits and bookmaker links here */>
 </div>
))
)}
 </div>
 </div>
);
};

export default ArbitrageDashboard;

```

## 3. Critical Production Pitfalls

If you push this to production right now, it will likely break in exactly one minute. Here is why:

- **Nginx Buffering:** If you use Nginx as a reverse proxy, it buffers HTTP responses by default. It will hold your SSE data until the buffer is full before sending it to the client, ruining your real-time latency. You must disable buffering for the SSE endpoint in your `nginx.conf`:

```
location /api/stream/ {
 proxy_pass http://localhost:8000;
 proxy_http_version 1.1;
 proxy_set_header Connection "";
 proxy_buffering off;
 proxy_cache off;
 proxy_read_timeout 86400s; # Prevent timeout disconnects
}
```

- **Browser Connection Limits:** Browsers limit HTTP/1.1 connections to the same domain (usually exactly 6). If a user opens 6 tabs of your dashboard, the 7th will freeze forever. Upgrading your production server to **HTTP/2** solves this, as it allows multiplexing hundreds of streams over a single connection.
- **State Management:** Do not try to hold the entire history of arbitrage events in the React state. When an event is resolved or expires (which happens in minutes), your backend should publish a "delete" or "expired" event with the Event ID so the frontend can remove it from the list.

*User prompt: How do I design the backend payload to tell the React frontend when an arbitrage opportunity is no longer available?*

Response: The most efficient way to handle arbitrage expirations over Server-Sent Events (SSE) is to leverage the native `event` field built into the SSE protocol, rather than trying to decipher every message through a single `data` stream.

By default, an SSE message just sends `data: { ... }`. However, the SSE specification allows you to prefix messages with an `event: type`. This allows your React frontend to listen to specific event channels rather than writing complex switch statements inside a generic `onmessage` handler.

## The SSE Payload Schema

You should define at least three distinct event types: `arb_new`, `arb_update`, and `arb_expired`.

When an arbitrage window closes (odds shift or the market suspends), your math engine should publish an `arb_expired` event containing only the unique identifier of the opportunity. There is no need to send the full payload to delete it.

### Raw SSE Network Stream:

```
event: arb_new
data: {"id": "arb_59382", "match": "Chiefs vs Pirates", "roi": 2.5}

event: arb_expired
data: {"id": "arb_59382", "reason": "odds_moved"}
```

## 1. FastAPI Backend Implementation

Modify your backend generator to parse the event type from your Redis Pub/Sub message and format it strictly to the SSE specification (separated by newline characters).

```
import asyncio
import json
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import redis.asyncio as redis

app = FastAPI()
redis_client = redis.from_url("redis://localhost:6379", decode_responses=True)

async def arb_event_generator():
 pubsub = redis_client.pubsub()
 # Subscribe to a channel where the Math Engine publishes all state changes
 await pubsub.subscribe("live_arbitrage_alerts")

 try:
 while True:
 message = await pubsub.get_message(ignore_subscribe_messages=True, timeout=1.0)

 if message:
 # Expecting the Math Engine to publish:
 # {"type": "arb_expired", "payload": {"id": "arb_59382"}}
 raw_data = json.loads(message["data"])
 event_type = raw_data.get("type", "message")
 payload = json.dumps(raw_data.get("payload", {}))

 # The exact SSE string format requires newlines
 yield f"event: {event_type}\ndata: {payload}\n\n"
 else:
 yield ": keep-alive\n\n"
```

```

 await asyncio.sleep(0.1)
 except asyncio.CancelledError:
 await pubsub.unsubscribe("live_arbitrage_alerts")

@app.get("/api/stream/arbitrage")
async def stream_arbitrage():
 return StreamingResponse(arb_event_generator(), media_type="text/event-stream")

```

## 2. React Frontend Implementation

On the frontend, replace the generic `eventSource.onmessage` with specific `addEventListener` hooks matching the event names you defined on the server.

```

import React, { useEffect, useState } from 'react';

const ArbitrageDashboard = () => {
 const [arbs, setArbs] = useState({}); // Use an object/dictionary for O(1) lookups and deletions

 useEffect(() => {
 const eventSource = new EventSource('http://localhost:8000/api/stream/arbitrage');

 // 1. Listen for new arbitrage opportunities
 eventSource.addEventListener('arb_new', (event) => {
 const newArb = JSON.parse(event.data);

 setArbs((prevArbs) => ({
 ...prevArbs,
 [newArb.id]: newArb // Store using the unique ID as the key
 }));
 });

 // 2. Listen for odds updates on existing arbs
 eventSource.addEventListener('arb_update', (event) => {
 const updatedArb = JSON.parse(event.data);

 setArbs((prevArbs) => {
 if (!prevArbs[updatedArb.id]) return prevArbs;
 return {
 ...prevArbs,
 [updatedArb.id]: { ...prevArbs[updatedArb.id], ...updatedArb }
 };
 });
 });

 // 3. Listen for expired arbs to remove them instantly
 eventSource.addEventListener('arb_expired', (event) => {
 const expiredArb = JSON.parse(event.data);

 setArbs((prevArbs) => {
 const nextArbs = { ...prevArbs };
 delete nextArbs[expiredArb.id]; // Instantly remove from UI
 return nextArbs;
 });
 });

 return () => eventSource.close();
 }, []);

 // Convert the dictionary back to an array for rendering
 const activeArbsList = Object.values(arbs);

 return (
 <div className="grid gap-4">
 {activeArbsList.map((arb) => (
 <div key={arb.id} className="bg-gray-800 p-4 rounded">
 <p>{arb.match} - {arb.roi}% ROI</p>
 </div>
))}
 </div>
);
};

export default ArbitrageDashboard;

```

## 3. Handling Silent Expirations

A critical failure point in streaming architectures is when an event expires, but the server fails to send the `arb_expired` message (e.g., the scraper crashes or the math engine stalls). The frontend will continue displaying an expired arb indefinitely, causing the user to place bad bets.

Implement a **frontend garbage collector**. When your server sends `arb_new`, include an `expires_at` timestamp. Have a `React useEffect` interval run every 5 seconds to sweep the state and automatically delete any arbitrage opportunity that has lived past its timestamp, regardless of whether the server explicitly cancelled it.

## Sources

1. [Using server-sent events - Web APIs | MDN](#)
2. [Server-Sent Events \(SSE\) - FastAPI](#)

*User prompt: How do I write a React useEffect interval to automatically delete stale arbitrage state data?*

**Response:** To build a reliable frontend garbage collector, you must set up an independent `useEffect` that periodically sweeps your state for expired timestamps.

The two critical requirements here are using a **functional state update** (so the interval doesn't get trapped in a stale closure) and **bailing out of re-renders** if nothing was actually deleted, ensuring your UI doesn't needlessly flash every 5 seconds.

Here is the implementation:

```
import React, { useEffect, useState } from 'react';

const ArbitrageDashboard = () => {
 const [arbs, setArbs] = useState({});

 // 1. Your SSE Connection (from previous step)
 useEffect(() => {
 // ... EventSource setup ...
 }, []);

 // 2. The Garbage Collector
 useEffect(() => {
 const SWEEP_INTERVAL_MS = 5000; // Check every 5 seconds

 const sweepInterval = setInterval(() => {
 const now = Date.now();

 setArbs((prevArbs) => {
 let hasChanges = false;

 // Shallow copy the previous state
 const nextArbs = { ...prevArbs };

 for (const [id, arb] of Object.entries(nextArbs)) {
 // Check if the current time has passed the arb's expiration timestamp
 // The backend should send `expires_at` as a Unix timestamp in milliseconds
 if (arb.expires_at && arb.expires_at < now) {
 delete nextArbs[id];
 hasChanges = true;
 }
 }

 // CRITICAL: If nothing was deleted, return the exact previous state object.
 // This tells React to skip the re-render entirely, saving CPU and battery.
 return hasChanges ? nextArbs : prevArbs;
 });
 }, SWEEP_INTERVAL_MS);

 // Cleanup the interval when the component unmounts
 return () => clearInterval(sweepInterval);
 }, []);

 const activeArbsList = Object.values(arbs);

 return (
 <div className="grid gap-4 p-4">
 {activeArbsList.map((arb) => (
 <div key={arb.id} className="bg-gray-800 p-4 rounded text-white border border-gray-700">
 <p className="font-bold text-blue-400">{arb.match}</p>
 <p className="text-green-400">{arb.roi}% ROI</p>
 <p className="text-xs text-gray-500 mt-2">
 Expires in: {Math.max(0, Math.floor((arb.expires_at - Date.now()) / 1000))}s
 </p>
 </div>
))}
 </div>
);
};
```

```
};

export default ArbitrageDashboard;
```

## Backend Requirement

For this to work flawlessly, your FastAPI backend must inject the `expires_at` field into the `arb_new` payload before sending it.

Do not rely on the React app to determine how long an arb should live. The math engine knows best. For example, a live in-play soccer arb might get a 15-second lifespan, while a pre-match rugby arb might get a 5-minute lifespan.

```
import time

Inside your Python Math Engine
payload = {
 "id": "arb_59382",
 "match": "Chiefs vs Pirates",
 "roi": 2.5,
 # Current Unix time in milliseconds + 30 seconds
 "expires_at": int((time.time() + 30) * 1000)
}
```

## Mitigating UI "Flicker"

Because sports betting odds fluctuate rapidly, a bookmaker might drop their odds (killing the arb) and raise them back up 2 seconds later. If you delete and recreate the arb in React that fast, the UI row will vanish and reappear, making it impossible for the user to click the bet links.

To fix this, you can implement a "Grace Period" UI state. When an arb expires or receives an `arb_expired` event from the server, don't delete it immediately. Instead, tag it with `status: 'expired'` and keep it in the DOM for 5 seconds with a grayed-out CSS class or a loading spinner. If the odds come back within 5 seconds, simply flip the status back to `'active'`.

*User prompt: How do I add JWT authentication to an EventSource connection if the native browser API doesn't support custom headers?*

**Response:** The native EventSource API strictly forbids custom headers. You cannot inject an `Authorization: Bearer <token>` header into it, which breaks standard JWT architectures.

To secure your SSE stream, you must use one of three methods, ranked from best to worst for a modern React/FastAPI stack.

## Method 1: The Fetch-Event-Source Wrapper (Recommended)

Microsoft maintains an open-source library called `@microsoft/fetch-event-source`. It completely replaces the native EventSource browser API by using the modern fetch API under the hood to consume the stream. Because it uses `fetch`, you have full access to custom headers.

### 1. Install the library:

```
npm install @microsoft/fetch-event-source
```

### 2. React Implementation:

```
import React, { useEffect, useState } from 'react';
import { fetchEventSource } from '@microsoft/fetch-event-source';

const ArbitrageDashboard = ({ jwtToken }) => {
 const [arbs, setArbs] = useState({});

 useEffect(() => {
 // AbortController allows us to close the connection on unmount
 const abortController = new AbortController();

 const connectStream = async () => {
 await fetchEventSource('http://localhost:8000/api/stream/arbitrage', {
 method: 'GET',
 headers: {
 'Authorization': `Bearer ${jwtToken}`,
 'Accept': 'text/event-stream',
 },
 signal: abortController.signal,

 // Fired on every new event
 onmessage(event) {
 const parsedData = JSON.parse(event.data);

 if (event.event === 'arb_new') {
 setArbs(prev => ({ ...prev, [parsedData.id]: parsedData }));
 }
 if (event.event === 'arb_expired') {
 setArbs(prev => {
 const next = { ...prev };
 });
 }
 }
 });
 };

 connectStream();
 }, [jwtToken]);
}
```

```

 delete next[parsedData.id];
 return next;
 });
}
},

// Handle token expiration or unauthorized errors
async onresponse(response) {
 if (response.status === 401) {
 console.error("JWT Expired");
 // Trigger your logout or token refresh logic here
 abortController.abort();
 }
},

onerror(err) {
 console.error("SSE Error:", err);
 // Return a number to dictate retry delay, or throw to stop retrying
 return 5000;
}
});
};

connectStream();

return () => abortController.abort();
}, [jwtToken]);

// ... render logic
};

```

*FastAPI requires absolutely no changes for this method. Your standard Depends(get\_current\_user) Bearer token logic will work perfectly.*

## Method 2: HttpOnly Cookies

If you prefer to stick to the native EventSource API and avoid external dependencies, you must move your JWT out of localStorage and into an HttpOnly cookie.

By adding withCredentials: true, the browser will automatically attach the cookie to the SSE request.

### React:

```

const eventSource = new EventSource('http://localhost:8000/api/stream/arbitrage', {
 withCredentials: true
});

```

**FastAPI:** Your backend must be configured to read the JWT from the cookie rather than the Authorization header, and your CORS configuration must explicitly allow credentials.

```

from fastapi import Cookie, HTTPException

```

```

async def get_current_user_from_cookie(access_token: str = Cookie(None)):
 if not access_token:
 raise HTTPException(status_code=401, detail="Not authenticated")
 # Verify JWT here...

```

## Method 3: The Short-Lived Ticket (URL Params)

You can technically pass a JWT in the URL: new EventSource(url?token=YOUR\_JWT). **Do not do this with your primary session JWT.** URLs are saved in browser history, proxy server logs, and AWS/Nginx access logs, making it a massive security vulnerability.

If you must use URL parameters, implement a **Ticketing System**:

1. Your React app makes a standard authenticated POST request (with the Bearer header) to /api/stream/ticket.
2. FastAPI verifies the Bearer token and generates a highly restricted, single-use JWT "ticket" that expires in 15 seconds.
3. React takes that short-lived ticket and opens the SSE connection: new EventSource('/api/stream/arbitrage?ticket=' + shortLivedTicket).
4. FastAPI verifies the ticket, opens the stream, and invalidates the ticket. Once the connection is established, it stays open indefinitely.

*User prompt: How do I handle JWT refresh tokens seamlessly if the SSE connection drops due to an expired token?*

**Response:** To handle seamless token refreshes with an SSE connection, you must prevent the stream library from aggressively auto-retrying with the expired token.

Because @microsoft/fetch-event-source gives you access to the HTTP response headers before the stream begins, you can intercept the 401 Unauthorized status, completely abort the current stream, trigger your refresh logic, and rely on React's lifecycle to open a new stream

once the fresh token is injected.

Here is the complete implementation using a standard React context or state management approach.

## 1. Define a Custom Error

Create a custom error class so your error handler knows exactly why the connection failed and doesn't confuse a 401 with a standard network timeout.

```
class UnauthorizedError extends Error {
 constructor(message) {
 super(message);
 this.name = 'UnauthorizedError';
 }
}
```

## 2. The React Implementation

In this pattern, the `useEffect` depends on `accessToken`. When the token expires, the stream throws the custom error and aborts. The `refreshAccessToken` function fetches a new token and updates the state. React sees the new `accessToken`, re-runs the `useEffect`, and seamlessly reconnects to the stream.

```
import React, { useEffect, useState, useRef } from 'react';
import { fetchEventSource } from '@microsoft/fetch-event-source';

const ArbitrageDashboard = ({ accessToken, refreshAccessToken }) => {
 const [arbs, setArbs] = useState({});
 const [isConnecting, setIsConnecting] = useState(true);

 // Use a ref to prevent overlapping refresh calls if multiple components fail at once
 const isRefreshingRef = useRef(false);

 useEffect(() => {
 if (!accessToken) return;

 const abortController = new AbortController();
 setIsConnecting(true);

 const connectStream = async () => {
 try {
 await fetchEventSource('http://localhost:8000/api/stream/arbitrage', {
 method: 'GET',
 headers: {
 'Authorization': `Bearer ${accessToken}`,
 'Accept': 'text/event-stream',
 },
 signal: abortController.signal,

 async onresponse(response) {
 // Intercept the 401 BEFORE the stream attempts to parse data
 if (response.status === 401) {
 throw new UnauthorizedError('JWT expired');
 }
 if (!response.ok) {
 throw new Error(`Server responded with ${response.status}`);
 }
 setIsConnecting(false);
 },

 onmessage(event) {
 const parsedData = JSON.parse(event.data);
 if (event.event === 'arb_new') {
 setArbs(prev => ({ ...prev, [parsedData.id]: parsedData }));
 }
 // ... handle arb_update and arb_expired ...
 },

 onerror(err) {
 if (err instanceof UnauthorizedError) {
 // 1. Abort the current EventSource connection immediately
 abortController.abort();

 // 2. Trigger the token refresh only once
 if (!isRefreshingRef.current) {
 isRefreshingRef.current = true;

 refreshAccessToken()
 .then(() => {
 isRefreshingRef.current = false;
 // The state update in refreshAccessToken will trigger

```

```

 // this useEffect to run again with the new token.
 })
 .catch((refreshErr) => {
 console.error("Refresh failed. Logging out.", refreshErr);
 // Trigger forced logout here
 });
 }

 // Throwing an error stops fetchEventSource from auto-retrying
 // with the bad token
 throw err;
 }

 // For standard network errors, return a number (ms) to auto-retry
 console.warn("Network error, retrying in 3s...", err);
 return 3000;
}
});
} catch (error) {
 // Catch the thrown UnauthorizedError so it doesn't crash the app
 if (error instanceof UnauthorizedError) {
 console.log("Stream paused for authentication refresh.");
 }
}
};

connectStream();

// Cleanup function when component unmounts or before re-running with a new token
return () => {
 abortController.abort();
};
}, [accessToken, refreshAccessToken]); // Effect re-runs when a fresh token arrives

return (
 <div className="p-4">
 {isConnecting ? (
 <div className="text-yellow-400">Connecting to live feed...</div>
) : (
 <div className="text-green-400">Live Feed Active</div>
)}
 { /* Render arbs */ }
 </div>
);
};

export default ArbitrageDashboard;

```

### 3. Backend Consideration: Graceful Disconnects

When your backend (FastAPI) realizes a token has expired mid-stream, it cannot simply return a 401 HTTP status code because the headers have already been sent when the connection was initially established (resulting in a 200 OK).

If your backend forces an expiration on a long-lived stream, it must push a specific SSE event instructing the client to disconnect and refresh:

#### FastAPI (Server-side expiration push):

```

event: auth_expired
data: {"reason": "token_lifetime_exceeded"}

```

**React (Client-side handler):** Add this alongside your `arb_new` logic inside the `onmessage` callback:

```

onmessage(event) {
 if (event.event === 'auth_expired') {
 abortController.abort();
 if (!isRefreshingRef.current) {
 isRefreshingRef.current = true;
 refreshAccessToken().finally(() => { isRefreshingRef.current = false; });
 }
 }
 // ... other events
}

```

This ensures that whether the token is expired at the exact moment of connection (handled by `onresponse`) or expires while the stream is actively open (handled by `onmessage`), the React app cleanly aborts, refreshes, and resumes without the user noticing anything more than a brief "Connecting..." indicator.

---

*User prompt: What is the best AWS architecture to deploy the Python scrapers, Redis, and FastAPI backend for low latency?*

Response: To achieve the lowest possible latency for an arbitrage platform, you must keep all components within the same Virtual Private Cloud (VPC) to eliminate public internet routing for internal traffic, and you must avoid AWS services that inherently buffer long-lived HTTP connections.

Here is the blueprint for deploying this stack on AWS.

## 1. The Core Infrastructure (VPC & Subnets)

Create a single VPC with **Public** and **Private** subnets across at least two Availability Zones (AZs) for redundancy.

- **Public Subnets:** House your NAT Gateways and the Application Load Balancer (ALB).
- **Private Subnets:** House your Scrapers, Redis, and FastAPI servers. Nothing here has a public IP address.

## 2. The Scrapers (Amazon ECS on Fargate)

Deploy your Python scrapers as Docker containers using **Amazon Elastic Container Service (ECS)** on **Fargate**.

- **Why Fargate:** Arbitrage scrapers often need to scale out horizontally during peak sports hours (e.g., Saturday afternoons) and scale in overnight. Fargate handles this container provisioning instantly without you managing EC2 instances.
- **Egress Traffic:** Route outbound scraping requests through NAT Gateways (in your public subnets). If bookmakers block your AWS IPs, configure your scrapers to route outbound requests through a third-party residential proxy service.

## 3. The Broker (Amazon ElastiCache for Redis)

Use **ElastiCache for Redis** deployed directly into your Private Subnets.

- **Instance Type:** Choose a compute-optimized node (like `cache.c6g.large`). Since Pub/Sub messages are not stored in memory after being dispatched, you do not need nodes with massive RAM; you need high network throughput.
- **Latency:** Because ElastiCache and your ECS containers share the same VPC, publish/subscribe latency is typically sub-millisecond.

## 4. The Backend (ECS on Fargate + Application Load Balancer)

Deploy your FastAPI application in Docker containers via ECS Fargate, placed in your Private Subnets. To route web traffic to them, put an **Application Load Balancer (ALB)** in front.

**The API Gateway Trap (Critical Warning):** Do *not* use Amazon API Gateway in front of your FastAPI SSE endpoint. API Gateway has a hard, unchangeable integration timeout of **29 seconds**. If no data flows for 29 seconds, API Gateway will forcefully terminate your SSE connection, causing a nightmare of reconnect loops on your React frontend.

**ALB Configuration for SSE:**

1. **Idle Timeout:** Increase the ALB's idle timeout from the default 60 seconds to a high value (e.g., 3600 seconds) so quiet periods in the sports schedule don't sever the connection.
2. **HTTP/2:** Enable HTTP/2 on the ALB. This allows the browser to multiplex the SSE stream alongside other API requests over a single TCP connection, bypassing the browser's 6-connection limit per domain.

## 5. Global Acceleration (Optional Edge Optimization)

If your servers are in `us-east-1` (Virginia) but your users are in Europe or Asia, standard internet routing will add 100-200ms of jitter to your SSE feed.

Attach **AWS Global Accelerator** to your ALB. This routes user traffic onto Amazon's private fiber-optic backbone at an edge location closest to the user, bypassing public ISP congestion. It drastically stabilizes persistent TCP connections like SSE.

*User prompt: If a bookmaker blocks AWS datacenter IPs, how do I integrate residential proxies into the Python ECS scrapers?*

Response: If a bookmaker's Web Application Firewall (WAF) flags and blocks AWS datacenter IP ranges, your scrapers will return 403 Forbidden or 1020 Access Denied errors instantly.

To bypass this, you must route all outbound traffic from your ECS Fargate tasks through a commercial residential proxy provider (such as Bright Data, Oxylabs, or Smartproxy).

Here is how to architect and implement this securely in your Python scrapers running on AWS.

## 1. Secure Credential Management via AWS ECS

Never hardcode proxy usernames and passwords into your Python code or Dockerfiles.

1. Store your proxy connection string (e.g., `http://customer-user:password@zproxy.lum-superproxy.io:22225`) in **AWS Systems Manager (SSM) Parameter Store** or **AWS Secrets Manager**.
2. Inject it into your ECS Fargate task definition as an environment variable:

```
"environment": [
 { "name": "TARGET_BOOKMAKER_URL", "value": "https://example-sportsbook.co.za" }
],
"secrets": [
 {
 "name": "PROXY_URL",
 "valueFrom": "arn:aws:ssm:af-south-1:123456789012:parameter/prod/proxy_url"
 }
]
```

## 2. Integrating Proxies into curl\_cffi (High-Speed Scrapers)

Inside your Python script, read the PROXY\_URL environment variable and pass it directly into your high-speed session client.

```
import os
from curl_cffi import requests

Read the secure proxy string injected by ECS
PROXY_URL = os.getenv("PROXY_URL")

proxies = {
 "http": PROXY_URL,
 "https": PROXY_URL
}

def fetch_with_proxy(api_endpoint, cookies, user_agent):
 session = requests.Session(impersonate="chrome124")

 # Assign the proxy dictionary to the session
 session.proxies = proxies

 response = session.get(
 api_endpoint,
 headers={"User-Agent": user_agent},
 cookies=cookies,
 timeout=10
)
 return response
```

## 3. Integrating Proxies into Playwright (Cloudflare Solvers)

If your scraper hits a JavaScript challenge and needs to spin up Playwright, the browser instance **must** use the exact same residential proxy. If Playwright solves the challenge on a residential IP but your curl\_cffi script requests the odds from an AWS IP, Cloudflare will instantly reject it.

```
import os
from playwright.async_api import async_playwright

PROXY_URL = os.getenv("PROXY_URL")

async def solve_challenge_with_proxy(target_url):
 async with async_playwright() as p:
 # Pass the proxy configuration directly to the browser launch options
 browser = await p.chromium.launch(
 headless=False,
 args=["--disable-blink-features=AutomationControlled"],
 proxy={"server": PROXY_URL} # Residential proxy enforced here
)

 context = await browser.new_context()
 page = await context.new_page()

 await page.goto(target_url, wait_until="domcontentloaded")
 # Extract cookies and user agent...
```

## 4. Managing Sticky Sessions (Crucial for Betting Sites)

Residential proxy providers rotate your IP address automatically every few requests. If your IP changes *while* you are navigating a sportsbook, the session cookies break, and you get banned.

- **The Solution:** Use **Sticky Sessions**. Most providers allow you to force a specific IP to stay active for a set window (e.g., 10 minutes) by appending a session tag to the proxy username or port.
- **Example (Bright Data format):**

```
http://customer-user-session-1928347:password@zproxy.lum-superproxy.io:22225
```

By appending `-session-[random_string]` to your username, the proxy provider locks that specific residential IP to your scraper task for the duration of that session, preventing sudden mid-request drops.

User prompt: Can you provide the Terraform configuration to deploy the ALB, ECS cluster, and ElastiCache Redis instance?

Response: Here is a complete, production-ready Terraform configuration to provision your core AWS infrastructure in the Cape Town region (af-south-1).

This setup deploys an ECS Fargate cluster, an Amazon ElastiCache Redis instance in private subnets, and an Application Load Balancer (ALB) explicitly configured with a high idle timeout for long-lived Server-Sent Events (SSE) connections.

### 1. Provider and Variables (provider.tf)

```
terraform {
 required_version = ">= 1.5.0"
 required_providers {
 aws = {
 source = "hashicorp/aws"
 version = "~> 5.0"
 }
 }
}

provider "aws" {
 region = "af-south-1" # Cape Town Region for lowest latency to SA bookmakers
}
```

### 2. ECS Cluster (ecs.tf)

```
resource "aws_ecs_cluster" "arbitrage_cluster" {
 name = "arbitrage-scanner-cluster"

 setting {
 name = "containerInsights"
 value = "enabled"
 }
}
```

### 3. ElastiCache Redis (redis.tf)

```
Subnet group for Redis inside your private subnets
resource "aws_elasticache_subnet_group" "redis_subnet" {
 name = "arbitrage-redis-subnets"
 subnet_ids = var.private_subnet_ids # Pass your private subnet IDs here
}

Security group allowing internal VPC traffic only
resource "aws_security_group" "redis_sg" {
 name = "arbitrage-redis-sg"
 description = "Allow inbound Redis traffic from VPC"
 vpc_id = var.vpc_id

 ingress {
 from_port = 6379
 to_port = 6379
 protocol = "tcp"
 cidr_blocks = [var.vpc_cidr]
 }

 egress {
 from_port = 0
 to_port = 0
 protocol = "-1"
 cidr_blocks = ["0.0.0.0/0"]
 }
}

resource "aws_elasticache_replication_group" "redis" {
 replication_group_id = "arbitrage-redis-cache"
 description = "Low-latency in-memory cache for live odds and pub/sub"
 node_type = "cache.c6g.large" # Compute-optimized node for throughput
 num_cache_clusters = 1
 port = 6379
 subnet_group_name = aws_elasticache_subnet_group.redis_subnet.name
 security_group_ids = [aws_security_group.redis_sg.id]
 automatic_failover_enabled = false
}
```

### 4. Application Load Balancer & SSE Target Group (alb.tf)

```

Public Security Group for ALB
resource "aws_security_group" "alb_sg" {
 name = "arbitrage-alb-sg"
 description = "Allow public HTTP/HTTPS traffic"
 vpc_id = var.vpc_id

 ingress {
 from_port = 80
 to_port = 80
 protocol = "tcp"
 cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
 from_port = 443
 to_port = 443
 protocol = "tcp"
 cidr_blocks = ["0.0.0.0/0"]
 }

 egress {
 from_port = 0
 to_port = 0
 protocol = "-1"
 cidr_blocks = ["0.0.0.0/0"]
 }
}

resource "aws_lb" "alb" {
 name = "arbitrage-alb"
 internal = false
 load_balancer_type = "application"
 security_groups = [aws_security_group.alb_sg.id]
 subnets = var.public_subnet_ids # Pass your public subnet IDs here

 # CRITICAL FOR SSE: Increase idle timeout to 1 hour (3600s) so quiet periods don't drop connections
 idle_timeout = 3600
}

resource "aws_lb_target_group" "fastapi_tg" {
 name = "fastapi-sse-tg"
 port = 8000
 protocol = "HTTP"
 vpc_id = var.vpc_id
 target_type = "ip"

 health_check {
 path = "/health"
 matcher = "200"
 interval = 15
 timeout = 5
 healthy_threshold = 2
 unhealthy_threshold = 3
 }
}

resource "aws_lb_listener" "http" {
 load_balancer_arn = aws_lb.alb.arn
 port = 80
 protocol = "HTTP"

 default_action {
 type = "forward"
 target_group_arn = aws_lb_target_group.fastapi_tg.arn
 }
}

```

### Key Production Configurations Included:

- idle\_timeout = 3600 on the ALB:** This prevents AWS from forcefully terminating your Server-Sent Events (SSE) stream if sports events experience a lull in odds updates.
- cache.c6g.large (Graviton2):** Provides high network performance and low CPU latency for handling intense Redis Pub/Sub message broadcasting between your scrapers and the API.
- Strict Security Groups:** The ElastiCache Redis cluster is completely isolated inside private subnets, accessible only by resources within your VPC CIDR block.

User prompt: How do I write the ECS Fargate task definition JSON to deploy the FastAPI backend container?

Response: To deploy your FastAPI backend container on AWS ECS Fargate, you need a task definition JSON file. This acts as a blueprint for your container, specifying CPU/memory allocation, environment variables, port mappings, and logging.

Here is a complete, production-ready ECS Fargate task definition for your FastAPI backend, configured to connect to your ElastiCache Redis instance.

### FastAPI Task Definition (task-definition.json)

```
{
 "family": "arbitrage-fastapi-task",
 "networkMode": "awsvpc",
 "requiresCompatibilities": [
 "FARGATE"
],
 "cpu": "256",
 "memory": "512",
 "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
 "taskRoleArn": "arn:aws:iam::123456789012:role/arbitrageTaskRole",
 "containerDefinitions": [
 {
 "name": "fastapi-backend",
 "image": "123456789012.dkr.ecr.af-south-1.amazonaws.com/arbitrage-fastapi:latest",
 "essential": true,
 "portMappings": [
 {
 "containerPort": 8000,
 "hostPort": 8000,
 "protocol": "tcp"
 }
],
 "environment": [
 {
 "name": "REDIS_HOST",
 "value": "arbitrage-redis-cache.xxxxxx.ng.0001.af-south-1.cache.amazonaws.com"
 },
 {
 "name": "REDIS_PORT",
 "value": "6379"
 }
],
 "logConfiguration": {
 "logDriver": "awslogs",
 "options": {
 "awslogs-group": "/ecs/arbitrage-fastapi",
 "awslogs-region": "af-south-1",
 "awslogs-stream-prefix": "fastapi"
 }
 }
 }
]
}
```

### Critical Configuration Breakdown

- networkMode: "awsvpc"**: Required for Fargate. This gives your container its own elastic network interface (ENI) and a private IP address within your VPC, allowing it to communicate directly with your private ElastiCache Redis instance.
- cpu: "256" and memory: "512"**: These represent 0.25 vCPU and 512MB of RAM. For an asynchronous FastAPI application broadcasting SSE streams, this is plenty of compute power to handle hundreds of concurrent persistent connections.
- awslogs Driver**: Always configure CloudWatch logging (`/ecs/arbitrage-fastapi`) so you can debug client connection drops, Redis Pub/Sub issues, or unhandled exceptions in production.

### How to Register and Deploy via AWS CLI

Once you save the file, register it with ECS using the AWS CLI:

```
aws ecs register-task-definition --cli-input-json file://task-definition.json --region af-south-1
```

User prompt: How do I configure ECS Service Auto Scaling for the FastAPI backend based on network load or CPU utilization?

Response: To configure ECS Service Auto Scaling for your FastAPI backend based on CPU utilization or network load (such as handling high volumes of concurrent SSE connections), you need to set up three key Terraform resources: an **ECS Target Tracking Scaling Policy**, a **CloudWatch Metric Alarm**, and the **App Auto Scaling Target**.

While CPU utilization is standard, for an SSE streaming application, tracking **Request Count per Target** (Network Load via ALB) is often a much better trigger because long-lived HTTP streams keep CPU usage deceptively low while consuming memory and connection slots.

Here is the complete Terraform configuration to implement Auto Scaling for your FastAPI ECS Service.

### ECS Service Auto Scaling Terraform Configuration (scaling.tf)

```
1. Register the ECS Service as a Scalable Target
resource "aws_appautoscaling_target" "fastapi_target" {
 max_capacity = 10
 min_capacity = 2
 resource_id = "service/${aws_ecs_cluster.arbitrage_cluster.name}/${aws_ecs_service.fastapi_service.name}"
 scalable_dimension = "ecs:service:DesiredCount"
 service_namespace = "ecs"
}

2. Define CPU-Based Target Tracking Scaling Policy
resource "aws_appautoscaling_policy" "ecs_policy_cpu" {
 name = "fastapi-cpu-autoscaling"
 policy_type = "TargetTrackingScaling"
 resource_id = aws_appautoscaling_target.fastapi_target.resource_id
 scalable_dimension = aws_appautoscaling_target.fastapi_target.scalable_dimension
 service_namespace = aws_appautoscaling_target.fastapi_target.service_namespace

 target_tracking_scaling_policy_configuration {
 predefined_metric_specification {
 predefined_metric_type = "ECSServiceAverageCPUUtilization"
 }
 target_value = 70.0 # Scale out when average CPU exceeds 70%
 scale_in_cooldown = 300 # Wait 5 minutes before scaling down
 scale_out_cooldown = 60 # Scale up quickly within 1 minute of high load
 }
}

3. (Optional Alternative) ALB Request Count Target Tracking
Best for SSE apps where CPU remains low but concurrent client connections spike
resource "aws_appautoscaling_policy" "ecs_policy_requests" {
 name = "fastapi-request-autoscaling"
 policy_type = "TargetTrackingScaling"
 resource_id = aws_appautoscaling_target.fastapi_target.resource_id
 scalable_dimension = aws_appautoscaling_target.fastapi_target.scalable_dimension
 service_namespace = aws_appautoscaling_target.fastapi_target.service_namespace

 target_tracking_scaling_policy_configuration {
 predefined_metric_specification {
 predefined_metric_type = "ALBRequestCountPerTarget"
 # Format: app/load-balancer-name/1234567890abcdef/target-group/target-group-name/1234567890abcdef
 resource_label = "${aws_lb.alb.arn_suffix}/${aws_lb_target_group.fastapi_tg.arn_suffix}"
 }
 target_value = 1000.0 # Target 1,000 active requests/connections per Fargate task
 scale_in_cooldown = 300
 scale_out_cooldown = 60
 }
}
```

### Key Operational Considerations for SSE Apps

- Graceful Draining on Scale-In:** When ECS decides to scale down (reduce task count), it will terminate containers. If a container is actively streaming SSE data to clients, an abrupt termination will drop their connection. Ensure your FastAPI application handles `SIGTERM` signals gracefully by listening for shutdown events, notifying connected clients to reconnect elsewhere, or allowing a drain period before exiting.
- Choosing CPU vs. Request Count:** Because Server-Sent Events maintain a persistent, open HTTP connection per client, your CPU usage might stay low (e.g., 15%) while network connection limits are saturated. Using `ALBRequestCountPerTarget` is usually safer for real-time dashboards than CPU utilization.

*User prompt: How do I handle SIGTERM graceful shutdown in FastAPI so clients don't get abruptly disconnected during ECS scale-in events?*

Response: When ECS initiates a scale-in event or deploys a new version, it sends a `SIGTERM` signal to your container, waits for a designated grace period (default is 30 seconds), and then forcefully terminates the container with `SIGKILL`.

By default, long-lived Server-Sent Events (SSE) connections will block FastAPI from exiting cleanly, or worse, get brutally severed mid-stream, throwing network errors on the React dashboard.

To handle `SIGTERM` gracefully, you need a mechanism to **track active SSE connections**, broadcast a shutdown warning event to clients so they can immediately reconnect to another healthy instance via the load balancer, and let FastAPI drain safely.

### Implementation: Connection Manager & Lifespan Hook

You can manage this cleanly using FastAPI's modern `lifespan context manager` alongside a connection tracker.

```

import asyncio
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import redis.asyncio as redis

Track active client queues globally in memory for this container instance
active_clients = set()

@asynccontextmanager
async def lifespan(app: FastAPI):
 # --- 1. STARTUP PHASE ---
 print("FastAPI container started, ready for traffic.")
 yield

 # --- 2. SHUTDOWN PHASE (Triggered by SIGTERM) ---
 print("SIGTERM received. Initiating graceful SSE drain...")

 # Step A: Notify all currently connected SSE clients to reconnect elsewhere
 disconnect_notification = "event: server_shutdown\nndata: {\\"message\\": \\"Server scaling down, reconnecting...\\"}"

 # Push the message to all active client queues managed by this instance
 for client_queue in active_clients:
 await client_queue.put(disconnect_notification)

 # Step B: Give clients 2 seconds to receive the message and close their sockets cleanly
 await asyncio.sleep(2.0)
 print("Graceful drain complete. Shutting down container.")

app = FastAPI(lifespan=lifespan)
redis_client = redis.from_url("redis://localhost:6379", decode_responses=True)

@app.get("/api/stream/arbitrage")
async def stream_arbitrage():
 async def event_generator():
 # Create a dedicated queue for this specific client connection
 q = asyncio.Queue()
 active_clients.add(q)

 pubsub = redis_client.pubsub()
 await pubsub.subscribe("live_arbitrage_alerts")

 try:
 while True:
 # Wait concurrently for either a new Redis message OR a shutdown push
 redis_task = asyncio.create_task(pubsub.get_message(ignore_subscribe_messages=True, timeout=1.0))
 queue_task = asyncio.create_task(q.get())

 done, pending = await asyncio.wait(
 [redis_task, queue_task],
 return_when=asyncio.FIRST_COMPLETED
)

 # Cancel pending tasks to prevent memory leaks
 for task in pending:
 task.cancel()

 # Check if the shutdown sequence sent a message to this queue
 if queue_task in done:
 shutdown_msg = queue_task.result()
 yield shutdown_msg
 break # Break loop, closing the SSE connection gracefully

 # Otherwise, handle the standard Redis message
 message = redis_task.result()
 if message:
 arb_data = message["data"]
 yield f"event: arb_new\nndata: {arb_data}\n\n"
 else:
 yield ": keep-alive\n\n"

 finally:
 # Clean up tracking set when client disconnects or container shuts down
 active_clients.remove(q)
 await pubsub.unsubscribe("live_arbitrage_alerts")

 return StreamingResponse(event_generator(), media_type="text/event-stream")

```

## React Frontend Handling

On the React frontend, listen for that custom `server_shutdown` event using your SSE wrapper (`@microsoft/fetch-event-source`). When received, close the stream explicitly and let your automatic reconnection logic spin up a fresh connection. The Application Load Balancer (ALB) will automatically route the new connection to a surviving, healthy ECS container.

```
onmessage(event) {
 if (event.event === 'server_shutdown') {
 console.log("Server is scaling down. Reconnecting to load balancer...");
 abortController.abort(); // Cleanly close current stream
 return; // Will trigger onerror retry logic to establish a fresh connection
 }
 // Handle normal arbs...
}
```

## Final ECS Task Definition Adjustments

By default, ECS waits **30 seconds** before sending SIGKILL after SIGTERM. If your cleanup logic or active streams need slightly more time to drain, you can explicitly configure the stop timeout in your ECS Task Definition:

```
"stopTimeout": 35
```

## Sources

1. [Lifespan Events - FastAPI](#)

*User prompt: How do I set up a GitHub Actions workflow to automatically build and deploy these Docker containers to ECS Fargate?*

**Response:** To automate your deployment pipeline, you can use a GitHub Actions workflow that triggers on every push to your main branch. The workflow will authenticate with AWS, build your FastAPI Docker image, push it to Amazon Elastic Container Registry (ECR), and force an ECS rolling update so your new code goes live with zero downtime.

Here is a complete, production-ready GitHub Actions workflow configuration.

## GitHub Actions Workflow (.github/workflows/deploy.yml)

```
name: Deploy to Amazon ECS Fargate

on:
 push:
 branches:
 - main

env:
 AWS_REGION: af-south-1
 ECR_REPOSITORY: arbitrage-fastapi
 ECS_CLUSTER: arbitrage-scanner-cluster
 ECS_SERVICE: arbitrage-fastapi-service
 CONTAINER_NAME: fastapi-backend

jobs:
 deploy:
 name: Build, Push, and Deploy
 runs-on: ubuntu-latest

 steps:
 - name: Checkout Code
 uses: actions/checkout@v4

 - name: Configure AWS Credentials
 uses: aws-actions/configure-aws-credentials@v4
 with:
 aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
 aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
 aws-region: ${ env.AWS_REGION }

 - name: Login to Amazon ECR
 id: login-ecr
 uses: aws-actions/amazon-ecr-login@v2

 - name: Build, Tag, and Push Docker Image to ECR
 id: build-image
 env:
 ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
 IMAGE_TAG: ${ github.sha }
 run: |
 docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
 docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
 echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_OUTPUT
```

```

- name: Download Task Definition
 run: |
 aws ecs describe-task-definition --task-definition ${{ env.ECS_SERVICE }} --query taskDefinition > task-de

- name: Fill in the New Image ID in the ECS Task Definition
 id: render-task-def
 uses: aws-actions/amazon-ecs-render-task-definition@v1
 with:
 task-definition: task-definition.json
 container-name: ${{ env.CONTAINER_NAME }}
 image: ${{ steps.build-image.outputs.image }}

- name: Deploy Amazon ECS Task Definition
 uses: aws-actions/amazon-ecs-deploy-task-definition@v2
 with:
 task-definition: ${{ steps.render-task-def.outputs.task-definition }}
 service: ${{ env.ECS_SERVICE }}
 cluster: ${{ env.ECS_CLUSTER }}
 wait-for-service-stability: true

```

## Required GitHub Secrets Setup

To allow GitHub Actions to talk to your AWS account, you must configure two repository secrets in your GitHub project settings (Settings > Secrets and variables > Actions):

1. **AWS\_ACCESS\_KEY\_ID**: An IAM user access key with permissions to push to ECR and update ECS services.
2. **AWS\_SECRET\_ACCESS\_KEY**: The corresponding secret access key.

## Minimal IAM Policy for GitHub Actions

The IAM user associated with those keys needs a policy allowing these specific actions:

- `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:CompleteLayerUpload`, `ecr:UploadLayerPart`, `ecr:InitiateLayerUpload`, `ecr:PutImage`
- `ecs:DescribeTaskDefinition`, `ecs:RegisterTaskDefinition`, `ecs:UpdateService`
- `iam:PassRole` (required for passing the ECS execution/task roles)

*User prompt: How do I run Alembic database migrations automatically inside the GitHub Actions workflow before ECS updates the service?*

**Response:** To run Alembic database migrations automatically before your ECS service updates, the standard practice is to use an **ECS "Run Task"** command in your GitHub Actions workflow.

Instead of starting a long-running service, `run-task` spins up an ephemeral Fargate container using your exact Docker image, overrides the default command to execute `alembic upgrade head`, waits for it to complete successfully, and *then* proceeds to update your main ECS service. If the migration fails, the workflow aborts immediately, preventing broken code from hitting production.

Here is how to modify your GitHub Actions workflow to include the migration step.

## Updated GitHub Actions Workflow (.github/workflows/deploy.yml)

Add the migration job right after pushing the Docker image to ECR and before updating the ECS service.

```

name: Deploy to Amazon ECS Fargate with Migrations

on:
 push:
 branches:
 - main

env:
 AWS_REGION: af-south-1
 ECR_REPOSITORY: arbitrage-fastapi
 ECS_CLUSTER: arbitrage-scanner-cluster
 ECS_SERVICE: arbitrage-fastapi-service
 CONTAINER_NAME: fastapi-backend
 SUBNET_ID: subnet-0123456789abcdef0 # Private subnet with DB/internet access
 SECURITY_GROUP_ID: sg-0123456789abcdef0 # Database/ECS security group

jobs:
 deploy:
 name: Build, Migrate, and Deploy
 runs-on: ubuntu-latest

 steps:
 - name: Checkout Code
 uses: actions/checkout@v4

```

```

- name: Configure AWS Credentials
 uses: aws-actions/configure-aws-credentials@v4
 with:
 aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
 aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
 aws-region: ${ env.AWS_REGION }

- name: Login to Amazon ECR
 id: login-ecr
 uses: aws-actions/amazon-ecr-login@v2

- name: Build, Tag, and Push Docker Image to ECR
 id: build-image
 env:
 ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
 IMAGE_TAG: ${ github.sha }
 run: |
 docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
 docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
 echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_OUTPUT

- name: Download Task Definition
 run: |
 aws ecs describe-task-definition --task-definition ${ env.ECS_SERVICE } --query taskDefinition > task-de

- name: Fill in the New Image ID in the ECS Task Definition
 id: render-task-def
 uses: aws-actions/amazon-ecs-render-task-definition@v1
 with:
 task-definition: task-definition.json
 container-name: ${ env.CONTAINER_NAME }
 image: ${ steps.build-image.outputs.image }

--- NEW: RUN ALEMBIC MIGRATIONS ON A STANDALONE FARGATE TASK ---
- name: Run Database Migrations
 uses: geekcell/github-action-aws-ecs-run-task@v3.0.0
 with:
 cluster: ${ env.ECS_CLUSTER }
 task-definition: ${ steps.render-task-def.outputs.task-definition }
 subnet-ids: ${ env.SUBNET_ID }
 security-group-ids: ${ env.SECURITY_GROUP_ID }
 assign-public-ip: "DISABLED"
 override-container: ${ env.CONTAINER_NAME }
 override-container-command: |
 alembic upgrade head
 tail-logs: true
 task-wait-until-stopped: true

--- DEPLOY UPDATED SERVICE ONLY IF MIGRATIONS SUCCEED ---
- name: Deploy Amazon ECS Task Definition
 uses: aws-actions/amazon-ecs-deploy-task-definition@v2
 with:
 task-definition: ${ steps.render-task-def.outputs.task-definition }
 service: ${ env.ECS_SERVICE }
 cluster: ${ env.ECS_CLUSTER }
 wait-for-service-stability: true

```

## Important Prerequisites

- 1. Database Access from Fargate:** The task execution environment requires network access to your PostgreSQL database (e.g., Amazon RDS). Ensure the SUBNET\_ID and SECURITY\_GROUP\_ID specified in the workflow variables grant outbound access to your database's port (typically TCP 5432).
- 2. Container Entrypoint Configuration:** Ensure your FastAPI Docker container's Dockerfile or entrypoint doesn't hardcode a startup command that blocks or overrides runtime command overrides. If you use a CMD array (e.g., ["uvicorn", "app.main:app", "--host", "0.0.0.0"]), setting override-container-command: alembic upgrade head safely replaces it for this run-task execution only.
- 3. IAM Permissions for GitHub Actions:** The IAM user/role running the GitHub Actions workflow needs permission to call ecs:RunTask and iam:PassRole (to pass the task execution roles), in addition to the ECR and ECS update permissions configured previously.